Micropin 8085 Port-Peripheral Design

A small 8085 command shim lets a modern controller own game logic while the original CPU remains the electrically compatible interface to the pinball hardware.

Executive summary

Keep the real 8085, its clock, its bus timing, and the motherboard's existing I/O circuitry. Replace the packed game ROM with a deliberately small firmware program. That program exchanges command packets with a Pico W, Raspberry Pi, or similar controller through one newly added I/O peripheral. The modern controller implements gameplay, networking, diagnostics, and development tooling; the 8085 performs the board-native RAM and port operations.

The essential rule: the 8085 remains the bus master. The peripheral never drives the shared bus except when the 8085 has explicitly selected it during an I/O read.

Division of responsibilities

Component	Responsibilities
Modern controller	Game rules, scoring decisions, configuration, Wi-Fi/USB protocol, logging, UI, assets, persistent state, command queues.
8085 command firmware	Read commands, validate them, update physical RAM, issue existing OUT instructions, read existing switch inputs, acknowledge results, maintain watchdog/safety behavior.
New I/O interface	Decode one reserved port address, provide data/status registers and optional interrupt request, translate voltage levels, obey bus timing.
Existing Micropin hardware	Switch matrix, lamps, coils, displays, sound and any autonomous logic that observes shared RAM or normal I/O signals.

Why this is attractive

The original 8085 code is no longer responsible for fitting an entire game into 8 KB. Modern code can be written and tested in a higher-level environment. At the same time, the riskiest portion - cycle-level interaction with the original motherboard - remains ordinary 8085 instructions on the original processor.

System view

```
Modern game code
| Wi-Fi / USB / serial
Modern controller + I/O peripheral
| command/status port
8085 command firmware
| RAM writes and IN/OUT instructions
Micropin motherboard hardware
```

How 8085 port I/O works

Ports are selected by the CPU; a peripheral does not independently push bytes onto the 8085 bus. Input and output are named from the CPU's perspective.

Instruction	What the 8085 does	What the new peripheral does
IN PORT	Places the port number on the I/O address bus, declares an I/O-read cycle, and samples D0-D7.	If its address is selected, it drives one byte on D0-D7 only for this read cycle.
OUT PORT	Places the port number on the address bus, places A on D0-D7, and declares an I/O-write cycle.	If its address is selected, it latches the byte. It does not drive D0-D7.

The 8085 multiplexes its low address/data pins. Address information is present first and ALE identifies that phase; later those pins carry data. The interface can latch the address itself, or use the motherboard's already latched address bus if it is accessible.

Suggested register map

Choose a port address only after tracing the board's I/O decoder. The current PPM assembly uses inputs 0x00-0x05 and outputs 0x00-0x0A plus 0x0D-0x0F. A safe new device must decode a full address of its own and must not alias an existing board device.

Port	Direction	Meaning
CMD_STATUS	IN	Bit flags: command waiting, response space, controller online, fault/heartbeat.
CMD_DATA	IN	Next byte of an incoming command packet from the modern controller.
RESP_DATA	OUT	Response/acknowledgment byte sent from the 8085 to the controller.
RESP_STATUS	OUT	Command completion code, error state, or heartbeat.

The four registers may be separate port addresses or register offsets behind one peripheral block. A FIFO is preferable to a single byte because it prevents loss when commands arrive in bursts.

Getting the 8085's attention

Simplest: the 8085 polls CMD_STATUS in its main loop. Lower latency: the peripheral asserts an unused, correctly interfaced interrupt input; the 8085 interrupt handler drains the command FIFO. In both cases, the 8085 performs IN instructions to retrieve bytes.

Minimal command firmware

The replacement ROM needs only initialization, a command loop, a small packet parser, routines for known-safe hardware actions, and a watchdog/fault path. It can keep useful state in the existing RAM so any board logic that observes RAM still behaves normally.

Illustrative command set

Command	Payload	8085 action
WRITE_RAM	16-bit address, 8-bit value	Validate address is within an approved RAM window; execute a normal store.
WRITE_PORT	port, value	Allow only an approved set of board ports; execute OUT.
PULSE_COIL	coil ID, duration	Validate coil and duration; turn on, time it, and force turn off.
READ_SWITCHES	request ID	Read the existing input ports and return a compact response packet.
HEARTBEAT	sequence	Refresh the controller watchdog; an expired watchdog enters a safe idle state.

Example command loop

```
loop:
IN CMD_STATUS
ANI CMD_WAITING
JZ loop
IN CMD_DATA ; opcode
... read remaining bytes ...
CALL validate_and_execute
OUT RESP_DATA
JMP loop
```

RAM and display considerations

Some RAM is ordinary game state; other bytes may be used as score/display buffers or be observed by autonomous board logic. The PPM disassembly places named score and display-related fields roughly in 0x2360-0x23FE, but the physical display path still needs confirmation from the board. If external logic reads the RAM, normal 8085 writes are ideal: it sees exactly the same electrical activity it saw from the original firmware.

Do not initially let the modern controller write RAM directly. Direct writes require the controller to request HOLD, wait for HLDA, take ownership of address/data/control lines, perform a correctly timed memory write, release the bus, and handle 5 V translation. That is a later optimization, not needed for the command-shim design.

Safety and bring-up plan

Phase	Goal
1. Observe	Trace/measure I/O decode, RAM access, interrupt use, voltage levels, and coil drive behavior. Confirm a non-aliasing port address.
2. Read-only peripheral	Implement STATUS/DATA reads and a polled 8085 diagnostic ROM. Verify address decoding and no data-bus contention.
3. Safe outputs	Add lamp/display outputs first. Keep coil commands disabled or hard-limited.
4. Controlled coils	Add whitelist, maximum pulse duration, dead-man timeout, and a hardware-safe default if the controller resets.
5. Game layer	Move rules, configuration, and operator functions into the modern environment.

Open hardware questions

Before committing to a PCB, identify the motherboard's exact 8085 variant, I/O address latch/decoder, available interrupt line, RAM chips, display-driver/PROM connections, and logic voltage thresholds. The Pico/Raspberry Pi GPIO must never connect directly to 5 V TTL signals; use appropriate direction-controlled level translation and ensure the peripheral is high impedance unless selected.

Design conclusion: this port-peripheral approach preserves the original CPU's bus and timing behavior while removing the original ROM-size constraint from game development. It is the recommended first architecture for a modern Micropin software platform.